Experiment Queue — ARC WhestBench 2026 (Phase 2)

Updated: 2026-09-25 (translated to English; D24 V31c dead, D25 Strassen L6 dropped, Q20 dead-neuron compaction running) · Best graded: #332110 score 5.1284e-9 (raw 2.1046e-8) — v40fb (FB 1.25 + beta 1.2). Previous: #331787 5.137e-9. Plain chain on public MLP #0: final MSE 2.290e-8.
Channel oracle (true values fed into mean closure): variance -47%, kappa4 diagonal -30%, kappa3 diagonal -14%, all three combined -94%. Q5: closure is near-exact; the error is the fresh kappa3/kappa4 error at each layer, which accumulates through mean and covariance. Horizon oracle: kappa3 memory is very long — even keeping 10 ages, layer 15 still has 16% error; age contributions are roughly equal and orthogonal.

All the ideas worth testing, from any AI model (Astra, Fable, GPT...) or from our own work, are sorted here by priority. P1 = cheap and high potential, do now; P2 = after P1; P3 = expensive or conditional. Dead ideas (tested and failed) are listed last, so time isn't wasted testing them again.

Summary (open and running)

ID	Priority	Status	Topic	Cost
Q20	P1	Running	Drop dead neurons from the source machinery (compaction)	step 1: 0 u, ~20 min local
Q14	P2	Open	Mixed-coordinate readout: cheapening Q8's kappa4 term (keeping the third leg in the old coordinate)	17-34 u (Astra)
Q15	P2	Open	Finding the partner channel for the S=2 'mystery' (cheap replay)	~3 chain runs (offline) + ~1 u replay
Q16	P2	Open	Eigen-truncated basis at join (aug64) — 100-MLP verification	+0.5% util
Q13	P3	Open	Variance 'super-closure' (bivariate diagram table cost 4/5)	~1-3 u
Q9	P3	Open	Joint knob search	Kaggle CPU

P1 — Now (cheap, high potential)

Q20 Drop dead neurons from the source machinery (compaction)

Running

Source: Forum audit 25 Sep (Phase 1 teams SOX 18106, baltsat 18149, ely2sh 18176: skip work on inactive neurons)

What to do	Neurons with Phi ~ 0 contribute nothing to the next transport (their w1 column is ~0) and their own kappa3 rows barely affect the closure. Step 1 (accuracy oracle): set w1 := 0 for Phi < thr (experiments/variants/estimator_q20dead.py, knob Q20_DEAD_THR) and measure final MSE on the two teacher networks. Step 2 (only if step 1 is flat): compact the legs and products to the live neurons per layer (Strassen-compatible sizes), dropping dead rows on input and output.
Evidence / so far	Teacher statistics on MLP #0: Phi < 1e-3 for 6% (L8), 12% (L12), 17% (L16) of neurons; Phi > 1 - 1e-4 for 2% / 7% / 13%. Potential saving ~2 x dead fraction on each transport / contraction at deep layers, ~8-10% of the total (estimate, not measured).
Cost	step 1: 0 u, ~20 min local
Gate to pass	step 1: final MSE change <= +0.3% on both networks at thr >= 1e-3
Next step	read step 1 results
File	experiments/variants/estimator_q20dead.py

P2 — After P1

Q14 Mixed-coordinate readout: cheapening Q8's kappa4 term (keeping the third leg in the old coordinate)

Open

Source: Astra R9 §1-3

What to do	Instead of a separate projection per source, first form the sum of sources R = S Q^T, then multiply once by B^T; the newborn's identity leg saves one product. Just a reordering of the computation — accuracy stays the same.
Evidence / so far	By Astra's accounting, extra cost is 17-34 u (part of it not yet measured); break-even is 25 u (if the -8.8% holds). At 20 u the score is only +1.8% better, at 35 u it's 3.5% worse. Astra itself says: this is not a moonshot, and it doesn't even get close to the leaders' cost.
Cost	17-34 u (Astra)
Gate to pass	Full metered cost <= 25 u and reproduce the -8.8% exactly
Next step	First verify the identity at layer 14 (R_total B^T = D21, to 1e-5).
File	consultations/ai-assistant-suggestions/GPT Astra 6 Ultra/Round_9_Astra_Architecture_Assessment.pdf

Q15 Finding the partner channel for the S=2 'mystery' (cheap replay)

Open

Source: Astra R9 §5

What to do	Record the variance/kappa3/kappa4 trajectory for S = 1, 1.25, 2, then replay the mean closure with teacher values substituted into each channel in turn — see which channel's oracle shifts the optimum toward S=1.
Evidence / so far	Astra: explained by simple quadratic cancellation (E(t) = 1 + A t + B t^2, A = -0.034, B = 0.068 -> at t = 0.25, -0.43%, matching the measured -0.35..-0.8%). In other words the 'mystery' isn't really a mystery — the restored term is a genuine improvement, but another error masks it.
Cost	~3 chain runs (offline) + ~1 u replay
Gate to pass	Whichever channel's oracle shifts the optimum toward S=2 is the partner
Next step	If time permits.

Q16 Eigen-truncated basis at join (aug64) — 100-MLP verification

Open

Source: Own idea (moonshot A, V41)

What to do	At join, instead of the range finder: transported old basis + the joiner's 64 new directions, truncated to r = 384 via eigen-decomposition of the full core.
Evidence / so far	Two teacher networks: util +0.5%, MSE -1.0% / -1.1%, score -0.5% / -0.55%.
Cost	+0.5% util
Gate to pass	100-MLP paired score < 1 - 2 SE
Next step	Together with the grid's best setting, on Kaggle.
File	experiments/variants/make_v41join.py

P3 — Conditional / expensive

Q13 Variance 'super-closure' (bivariate diagram table cost 4/5)

Open

Source: GPT 5.6 Sol High R7 Q1

What to do	Adding more diagrams (degree <= 4 block) to V29's pair closure table.
Evidence / so far	Q5: with true inputs, the closure's remaining error is only ~9-13% (part of which is teacher noise) — so this path's maximum possible gain is limited.
Cost	~1-3 u
Gate to pass	If teacher-forced projected dvar drops >= 25%
Next step	Low priority.

Q9 Joint knob search

Open

Source: Own idea (K3z-B)

What to do	Coordinate search over score with all knobs together.
Evidence / so far	100-MLP result: the knobs sit at the score-optimum (for age2_8, qpass, accuracy gains come with an equal rate of cost increase) — little hope of a big gain from a joint search.
Cost	Kaggle CPU
Gate to pass	Paired gain on 100-MLP
Next step	After Q3's results come in.

Finished (success / dead)

Q17 Young tier (~104 u) at half the cost — Round 10 prompt

Dead

Source: Own idea (moonshot A)

What to do	Fewer products for age 1-4 source transport + contraction: use A = P a_b, a shared propagator factor across consecutive sources, closed form for age-1/2, batched contraction.
Evidence / so far	Cost ledger: young transport ~54 u + contraction ~50 u. One leader showed equal accuracy at half the cost. Own algebra (24 Sep): (1) dropping A-transport via A = P a_b — no gain, the (P o A) term needs A's entry-wise values so building A still costs n^3; (2) P_old = P_new M — also costs n^3 to build; (3) batching — FLOP count unchanged; (4) age-1 P = W is already exploited. The Hadamard product is full-rank (Khatri-Rao wall) — 4 n^3 per source-layer is the floor for this structure. Any solution would have to be outside this structure. | 24-25 Sep: Round 10 closed — Sonnet's exact contraction-order search + 4 outside answers: 193 square products (D21) in any order.
Cost	Research
Gate to pass	<= 55 u while keeping D21 relative error <= 2%

Q18a Finite-Horizon Query Response (h = 2): 2 layers of kappa3 memory in query-space, the rest via tree

Dead

Source: GPT 5.6 Sol R9 (FHQR-2)

What to do	Instead of a source bank, carry two global n x n response matrices H = K3[:, Q, Q] (Q = the query for the next 1 and 2 layers); fill the older part with a dressed tree.
Evidence / so far	Horizon oracle (Fable's dense source capture, matches the source-sum chain D21 to 1.5e-7): keeping only the last h ages and dropping the rest, D21 error — layer 15: h2 84%, h4 65%, h6 46%, h8 31%, h10 16%; layer 11: h2 72%, h8 10%. Each age contributes almost equally (~0.15-0.20), and the contributions [corrected 24 Sep: not orthogonal — the ages are coherent, sum of squares 0.24-0.49]. FHQR-2 is dead; per Sol's rule: leaders keep long memory but in a completely different, cheap coordinate. | Result (q18_k4_horizon.py, N=1e6, MLP#0): erasing all memory at L-h and running the actual network forward h steps with Gaussian inputs, κ4 level at the decisive neuron, L12: h=1/2/3/4/6 → 2/6/12/18/35%, L16: 3/2/5/9/20%; pattern correlation with teacher L12 0.23/0.40/0.53/0.67/0.78, L16 0.20/0.41/0.59/0.65/0.77; best-scale errG4 h=2: 0.90/0.99, h=6: 0.37/0.63 — chain 0.19/0.21. The last 2 steps' births carry only 2-6% of κ4; even at 6 steps only 20-35%. κ4's memory is as long as κ3's: the young κ4 carrier is dead; the κ4 lever would need the full κ4 source chain — closed on cost grounds.
Cost	~82-90 u (Strassen) claimed
Gate to pass	teacher-forced D21 error <= 2.5%
File	experiments/probes/q18_k4_horizon.py

Q1 Quartic kappa4 kernel (cheap version) — new computation for the kappa4 diagonal

Dead

Source: Astra R7 §3-4

What to do	Recompute the quartic response polynomial F0 fresh at every ReLU; its non-pair part plus the chain's pair part give the next layer's kappa4 diagonal. Global rule: k4 = 0.3 x chain + 0.7 x (P_chain + 1.25 x kernel non-pair).
Evidence / so far	Non-pair part's correlation 0.63-0.78 (two networks). End-to-end (in the mean closure): A=0.5,S=1.5 -> MLP #0 -6.4%, synthetic -2.6%; A=0.3,S=1.25 -> -5.0% / -1.5%. Without Pchain (only chain kappa4 + kernel) it doesn't work — double counting. Adding GPT Sol High R7's K3+C '3+2' term (cheap t_rep, via the chain's D21/D3) raises non-pair correlation to 0.73-0.83 (from 0.68-0.78), scale deficit to 1.16-1.87 (from 1.34-2.16) — but at the decisive neurons correlation only gains +0.01-0.04, and the G4-weighted error is unchanged (x0.925). Late-layer/decisive-neuron test (A=0.5,S=1.5): all layers all neurons -6.4% / -2.6% @ ~42 u; all layers decisive neurons -5.2% / +0.3% @ ~22 u; layer 8-16 -3.5% / -1.1% @ ~11.5 u; layer 11-16 -2.8% / -0.2% @ ~7 u. No setting shows a net gain in score — dead in its current form.
Cost	3 products/layer: ~18 u (+7%) restricted to the ~400 decisive neurons, ~26 u (+10%) for all
Gate to pass	Net ≈ 0 as it stands (gain 2.6-6.4% vs. cost 7-10%). To become worthwhile: halve the cost, or double the accuracy (Q6/Q8)
Next step	Revisit only if Q12 (211 hypertree) makes the kernel much more accurate.
File	experiments/probes/probe_astra7_k4.py, experiments/chain/a7_kernel_e2e.py

Q2 R7_FB_SCALE = 1.25 (slightly raising the feedback constant)

Success

Source: Fable R7

What to do	Half of two Edgeworth K x C terms was dropped during the port; restoring them fully (2.0) is worse, 1.25 is slightly better.
Evidence / so far	100 MLP, paired (base bit-identical): fb1.25 score x0.9963 +- 0.0008; fb1.5 x1.0001; fb1.25 + beta1.2 x0.9946 +- 0.0007 (better on 73% of networks), zero cost. Submitted: v40fb (#332110), estimated LB 5.109e-9. GRADED: #332110 score 5.1284e-9 (raw 2.1046e-8) — new best (previous 5.137e-9): -0.17%, less than the local estimate (-0.54%) (the hidden network differs). | FB x beta grid (100 MLP, paired vs. shipped 1.25/1.2, 24 Sep): fb1.15/b1.2 +0.10% +-0.03, fb1.35/b1.2 +0.00% +-0.03, fb1.15/b1.3 +0.12% +-0.05, fb1.35/b1.1 +0.08% +-0.05 — the shipped setting is best (flat peak); fb1.25/b1.1 +0.04% +-0.03, fb1.25/b1.3 -0.01% +-0.03 — no difference; grid finished.
Cost	0 u (just a constant)
Gate to pass	Paired gain on the 100-MLP suite > SE
Next step	Done.
File	experiments/variants/r7_estimator_fbscale.py

Q3 100-MLP knob eval (6 variants) — running on Kaggle

Success

Source: Own idea (K3z-B)

What to do	beta1.2, lam0.9, row0.25, qpass2_3, age2_8, qpass_2 — paired comparison against the current best on the same 100 MLPs.
Evidence / so far	Runner-b and c finished (100 MLP, paired, bit-identical deterministic base): row0.25 score x1.0003; qpass2_3 x1.0006; age2_8 raw x0.9910 (better on 79% of networks) but util x1.0097 -> score x1.0006; qpass_2 raw x0.9904 but util x1.083 -> score x1.073. None is submit-worthy — the chain sits at the score-optimum (1% accuracy = 1% cost). Harness SE +-0.01-0.15%. LB estimate = 5.137e-9 x paired score ratio (LB raw = local x 0.956). Kaggle wall median ~111 s, max ~134 s — a time risk for the extra compute. runner-a is running. runner-a: beta1.2 score x0.9993 +- 0.0005 (a small free improvement, being tested together with Q2); lam0.9 x1.0026 (worse). Base is bit-identical across the three runners -> no need to run base again.
Cost	Kaggle CPU (costs us no time)
Gate to pass	Submit if score (MSE x util) is significantly lower
Next step	Done. No single variant is submit-worthy on its own.
File	kaggle/runners/runner_{a,b,c}, kaggle/estimators/set_{a,b,c}

Q4 Oracle: true D21 only in the feedback leg

Dead

Source: Fable R7

What to do	55-65% of the kappa3 slice error is still unexplained; isolate whether it's closure cross-talk.
Evidence / so far	Fable: first-order theory explains 35-45%. Fable R8: ~55% of the 'all-distinct' kappa3 error remains unexplained even to second order — the floor of this formulation. | 25 Sep: run by Opus R12 — true D21 only in the feedback leg gives x0.9999 (no effect).
Cost	One chain run (offline oracle)
Gate to pass	If slice error or final MSE drops significantly -> look for a cheap approximation

Q5 Teacher-forced bivariate closure: which input creates the covariance error

Success

Source: Astra R7 §1-2

What to do	Run the closure at layers 8, 12, 15 with true cumulants; separately swap in only kappa31 / K22 / D21 to see which one reduces the fresh covariance error (4.2e-5).
Evidence / so far	Result (weighted next-layer variance error, layer 7 / 14): all inputs true -> 0.127 / 0.092; only mu, var, C true -> 0.376 / 0.185; only kappa3 true -> 0.715 / 0.853; only kappa4 true -> 0.960 / 0.998. Conclusion: the closure is near-exact; most of the covariance error (81% at depth) is accumulated mu/C error carried from earlier layers; the fresh error comes from kappa3 (15-28%) and the closure residual. There's no separate term to fix for variance — the fresh kappa3/kappa4 error at each layer has to be reduced.
Cost	<10 u offline
Gate to pass	If swapping any one input greatly reduces the projected variance error -> fixing that input becomes the target
Next step	Q10 (structure of E) is still meaningful; the main lever now is the fresh error in the kappa3 slice and kappa4 diagonal.
File	data/kaggle_out/kaggle_out_k4post

Q6 Adding a K4 x C term to the kernel

Dead

Source: Astra R7 appendix A

What to do	A next-order term combining the harmonic input kappa4 with covariance (two extra query products).
Evidence / so far	Q1's kernel underestimates magnitude at depth (needed scale 1.3 -> 2.2); this term could be part of that shortfall. | 25 Sep: Q1 is dead, and Opus R12 (D22) showed the cheap forms of the kernel are net ≤ 0; κ4's deep pattern is long memory (C15, Q18).
Cost	+2m/n u/layer (~+0.7 u per layer at m=350)
Gate to pass	If non-pair corr/slope improves in a one-layer diagnostic

Q7 Also taking the (3,1) slice from the kernel

Dead

Source: Astra R7 §4

What to do	kappa31 slice from the gradient of the same polynomial; could also affect the covariance channel.
Evidence / so far	Fable R8: the (3,1) sector is negligible (3e-5 within 1e-3).
Cost	~42 u total
Gate to pass	raw MSE must drop by >13.9%
Next step	If both Q1 and Q5 turn out positive.

Q8 K3 x C directional query (full version of the kernel)

Dead

Source: Astra R7 §5 + Fable R8

What to do	The K3 x C term missing from the kernel, reusing the source leg.
Evidence / so far	Fable R8: the kernel's depth shortfall is exactly this (1,1,1,1) K3 x C term — via the chain's own 'all-indices-distinct' kappa3 from its source (Sol's t_rep can't see this sector at all). Measured coefficient 1.0 +- 0.1 (theory 1); correlation at decisive neurons 0.69-0.75 -> 0.75-0.81; G4-weighted kappa4 error 0.095/0.116/0.157/0.193/0.197 (L4/8/12/15/16; chain 0.124/0.142/0.193/0.199/0.210). Age 1-4 covers ~90% of the source, age 1-2 ~65%. End-to-end (MLP #0, A=0.5,S=1.5 + term): -8.8% (2.0893e-8) — but cost ~140 u (+55%), not viable. Cheap version (MLP #0; the synthetic run stopped at a session restart): layer 8-16 + decisive neurons + 2 sources -> -4.2% @ ~35 u classical (+13%); all layers + decisive neurons + 2 sources -> -6.25% @ ~52 u; layer 8-16 without the term -> -3.5%. None shows a net gain in score — dead in the current architecture.
Cost	Full: ~140 u; cheap version (layer 8-16, |alpha|<1.5, 2 new sources): ~26 u classical / ~13 u Strassen
Gate to pass	If the cheap version drops raw MSE > 9% (classical) or > 5% (Strassen)
Next step	Reconsider only in a new architecture (folding the term inside the transport product).
File	experiments/probes/r8_k4_1111.py, r8_k4_dist_e2e.py; consultations/fable/ROUND8_fable_synthesis.md

Q12 Adding nine cheap (2,1,1) hypertree classes to the kernel

Dead

Source: GPT 5.6 Sol High R7 Q2.3

What to do	Of the 12 symmetry classes in the (2,1,1) part of the birth, 9 are hypertrees — computable as n x m messages via C, C.^2, D21, D21^T; the 3 non-hypertree classes (triangle etc.) are excluded.
Evidence / so far	Fable R8: exact (2,1,1) law + Sol's nine hypertrees + triangle: fixed-coefficient R2 <= 0.07, worse at every layer for the decisive neurons.
Cost	+5-15 u (late-layer version)
Gate to pass	If total kappa4 residual RMS drops >= 40%; otherwise drop it
Next step	A weighted, capped evaluator using the graph list from ARC's factor_k4.py / TERM_TABLE_211.

Q10 Is the covariance error E low-rank in the direction of the target neurons?

Dead

Source: GPT 5.6 Terra R7 Q1.2

What to do	E = C_true_post - C_chain_post; its eigen-decomposition; how much of the variance correction w_i^T E w_i for the ~350 decisive neurons can be captured with the top-k directions (k = 4, 8, 16, 32). Whether the top directions align with any known vector (mean, C's eigenvector).
Evidence / so far	E's top-8 directions hold 26-49% of the energy, and they align with the post-ReLU mean vector (overlap 0.75-0.79) and C's top eigenvector (0.68-0.82) — a coherent error. But the decisive neurons' (|alpha|<0.75) variance correction comes from the random, full-rank part: error at k=32 is 0.81-1.04, k=256 is needed. Low-rank correction doesn't work.
Cost	Offline diagnostic only (a few minutes)
Gate to pass	0.1% relative accuracy at k <= 32, and the directions predictable from some known quantity in the chain
Next step	After Q5 is done.
File	experiments/probes/q10_error_rank.py

Q11 Monte Carlo with a coupled surrogate — how much variance actually drops (kill-test)

Dead

Source: Astra R7 §6 / GPT Terra R7 Q1.4 / Grok R7 1c

What to do	A control variate using a linearized network G(x) = m + J x (coupled via the same x); measure the variance reduction factor for the variance/kappa3/kappa4 correction with ~2000 samples.
Evidence / so far	Correlation with the coupled linear surrogate (variance statistic): layer 2 -> 0.73, 4 -> 0.50, 8 -> 0.32, 16 -> 0.19; variance drops only x1.0-2.2 (x100 was needed). The sampling path is closed; Grok's 40x claim is disproved.
Cost	A few minutes on CPU (numpy)
Gate to pass	Only proceed if reduction >= 100x
Next step	A quick kill-test if time permits.
File	experiments/probes/q11_surrogate_cv.py

D1 Covariance patch (C^3, C^4, C x kappa31 term)

Dead

Source: Astra R7 §2

What to do	Three terms missing from V29's (1,1) closure.
Evidence / so far	The terms are very small (rms 2-3e-6 vs. error 6.5-9e-5), correlation with the error ~0, projected variance error x0.998-1.007.
Cost	<0.2 u
File	experiments/probes/probe_astra7_cov.py

D2 Three quantities via Monte Carlo / control variate

Dead

Source: Astra R7 §6

What to do	The variance/kappa3/kappa4 correction via sampling.
Evidence / so far	Theory: reducing variance 100x needs surrogate correlation >= 0.997; 10,000 samples = 156 u, and even then variance error is 1.4%.

D3 kappa3 transport curvature, exact old tier, R_FB 128, FB_SCALE 2.0

Dead

Source: Fable R7 / own idea

What to do	Attempt to reduce the kappa3 slice error.
Evidence / so far	No coherent component in the curvature; exact old tier -3.9% MSE / +19% cost; R_FB 128 -3.1% / +43%; FB_SCALE 2.0 +3.4% worse.
File	consultations/fable/ROUND7_fable_report.md

D4 Cheap paths for the kappa4 diagonal: V39 multiplier, pairwise, 13-feature, star source, |kappa3|

Dead

Source: Own idea (23 Sep)

What to do	Attempt to fix the kappa4 diagonal.
Evidence / so far	V39 16-26% worse; pairwise correlation 0.24-0.59; feature 0.19->0.18; star 10% at depth; |D3| only on non-decisive neurons.
File	docs/build_notes.py K3ab-K3ad

D5 Grok's D21-scaling covariance term, Grok's pairwise kappa4 (6 w2 K22 w2 + 4 w3 K31 w), Copilot's Hutchinson probe and aggregate matrix

Dead

Source: Grok R7 1b, Q2 / Copilot R7

What to do	Old proposals of the same kind.
Evidence / so far	Scaling of the D21-type term: closure-term regression R2 <= 0.08 (R27); pairwise kappa4 correlation is 0.24-0.59 (K3ad) even with the true post slice; Hutchinson needs the unknown error E; the aggregate matrix family is already dead (K3e).
File	consultations/ai-assistant-suggestions/{Grok,Copilot}/*Round*7*

D6 Random-weight mixture memo; Gemini's kappa3 x kappa3 covariance term; quadratic surrogate MC

Dead

Source: DeepSeek R7 / Gemini R7 Q1, Q3

What to do	(a) A Gaussian mixture 'assuming random weights': kappa3_iii = 0, 300 samples suffice; (b) a sum_ab k3(j,a,b) k3(k,a,b) term in Cov; (c) a 2nd-order surrogate control variate.
Evidence / so far	(a) Our weights are fixed; teacher kappa3_iii rms is ~1e-2, and the chain matches it at slope 1.00 — the claim is wrong. (b) Cov(relu z_j, relu z_k) depends only on the joint distribution of (z_j, z_k) — no other index can enter; Q5 shows the closure is exact with true inputs. (c) Measured: correlation layer 2 -> 0.967 (x15), 4 -> 0.889, 8 -> 0.733, 12 -> 0.454, 16 -> 0.00 — breaks down at depth.
File	experiments/probes/q11_surrogate_cv.py (SURR=quad)

D7 DeepSeek 7.1: 'missing (2,1) kappa3 x kappa3 covariance term', sparse pair-neighbour variance, rank-R kappa4 CP

Dead

Source: DeepSeek R7.1

Evidence / so far	The kappa3 x kappa3 term (d21T*d21T, d21*d21T) is already in the chain's (1,1) table, and Q5 shows the closure is exact; the error is spread across the whole matrix (Q10) so a sparse neighbour approach won't work; rank-R kappa4 CP is the already-dead aggregate/Tucker family (K3e, K3j).
File	consultations/ai-assistant-suggestions/DeepSeek, GPT 5.6 Terra

D8 Variance adjoint recursion, top-5% stratified MC, 'closed-form' (2,1,1)/(1,1,1,1) kappa4, Rao-Blackwell MC

Dead

Source: Unknown AI (pasted into chat, 23 Sep)

What to do	Four proposals.
Evidence / so far	(1) computing variance from the chain's own C — doesn't improve accuracy, and a single adjoint vector can't represent many quadratic forms; (2) sampling the unknown error ΔC is impossible, and the error is full-rank (Q10); (3) the formula is wrong — it writes (1,1,1,1) as sum w^4 kappa4_jjjj (that's the diagonal term, already in the chain), while (2,1,1) has a single index; (4) assumes random weights (ours are fixed), the '30x' claim is baseless (Q11: x1-2), and the cost is off by 1000x.

D9 kappa3's second order (K x K), the coherent part of the D21 error, kappa4 memory (old star), non-Gaussian vertex weight

Dead

Source: Fable R8 (our own test)

What to do	Searching for the source of the variance/kappa4 channel.
Evidence / so far	The 12 terms of K x K: rms 3e-6-2e-5 vs. error 2.2e-4 (negligible; this also rules out R7's explanation for 'why S=2 is worse', no XY knob needed); the rank-1 (mu-direction) part of the D21 error doesn't even see the decisive neurons; inversely correlated with the old star birth residual (coef -0.9); vertex weight ratio 1.00.
File	experiments/probes/r8_kk_terms.py, r8_k4_memory.py, r8_k4_ngvertex.py, r8_dist_check.py

D10 Explicit all-distinct kappa3 tensor, reverse cubic-functional circuit, speeding up only the young tier

Dead

Source: Astra R9 §4 (derived)

What to do	Three possible directions for a new architecture.
Evidence / so far	Packed kappa3 tensor 0.72 GB, transport ~1567 u/layer; in circuit form, again ~2 u per source-layer; even eliminating the entire young tier (116 u) leaves 144 u — to reach 150 u with the kappa4 fix included, more than the young tier itself would need to be saved.

D11 Dropping the range finder at join (transport mode) and a smaller sketch (aug16/32)

Dead

Source: Own idea (moonshot A, V41)

What to do	Reducing the cost of join.
Evidence / so far	transport: util -6.6% but MSE +6.3% / +1.0%; aug16/32: score +1.3% / +0.8% worse. No large cost savings from join.
File	experiments/variants/make_v41join.py

D12 DHMP: drop the source bank, get the 'all-indices-distinct' kappa3 per layer from a dressed tree

Dead

Source: GPT 5.6 Sol R9 (DHMP)

What to do	A source-free architecture (claimed ~74-90 u).
Evidence / so far	Kill gate (teacher-forced, true post E, T, d3): next-layer D21 relative error 40.4% / 38.4% / 36.4% / 36.7% / 36.3% (L4/8/12/15/16) — required <= 2.5%, chain gets 4-7%. Correlation with the tree's 'all-indices-distinct' part is only 0.43-0.54 (scale correct, slope 0.88-1.16): the other half is memory from earlier layers. Dressed vs. bare edge difference 0.4-1% — doesn't explain the paradox either.
File	experiments/probes/r9_dhmp_gate.py

D13 Compressing kappa3 memory along the age axis (age-mode projection)

Dead

Source: GPT 5.6 Sol (Age-Gram test)

What to do	Reconstructing total D21 from the first r modes of the Gram matrix of all ages' D21 contributions.
Evidence / so far	Correction: the ages are not orthogonal, they're coherent (sum s^2 = 0.25-0.49, mean cosine 0.29-0.39, first mode 34-47% of energy, all loadings the same sign). But total D21 error: layer 15 r1 12.2%, r4 10.3%, r6 8.5%; layer 14 r1 15.4%, r6 9.7%; layer 11 r1 23.5%, r6 7.4% — far above the 2% target. At depth the first mode captures ~88% of the total, the remaining ~12% spread evenly across many modes.
Next step	Whether the first mode transports as a single object from layer to layer — a question for Sol.
File	experiments/probes/r9_age_gram.py

D14 Carrying the first age-mode as a single object from layer to layer (S_{l+1} = rho T S_l + birth)

Dead

Source: GPT 5.6 Sol (shift-stable coherent carrier)

What to do	Summing all sources weighted by the Age-Gram's first mode into a single carrier; whether its weight shifts by one step at the next layer.
Evidence / so far	L14 -> L15 (with the true transported source): rho* = 0.944, eps_shift = 2.9% (required <= 2%; 2-5% = 'backbone'). Total D21 via carrier + birth: 13.9% error. Main point: any fixed weight transports under the same linear map, and a uniform weight (the total kappa3 itself) is exactly shift-stable — so the coherent mode adds nothing new along the time axis. The problem is entirely SPATIAL: capturing a dense cubic tensor (the total) cheaply.
Next step	Spatial question: what representation of the total kappa3 is cheaper than 3kn columns and survives transport + ReLU? (Tucker/TT/ISDF/vector-metric are dead)
File	experiments/probes/r9_shift_carrier.py

D15 Any linear compression of total kappa3 that is closed under transport; lazy cubic circuit

Source: GPT 5.6 Sol (irreducibility + lazy cubic circuit)

Dead

What to do	(1) Proof: for every dense M, the dimension of a closed linear state is >= C(n+2,3) ~ 1.8e8 (Sym^3 irreducible). (2) D21 in reverse mode, keeping only the birth matrix + M.
Evidence / so far	Circuit computation: ~4l square products at depth l, ~480 total, ~245 u — equal to the chain's 249 u (two contraction orders of the same tensor network). Our correction: M is fixed and known, so the real bound is what future readout can see (<= n^2 per remaining layer) — close to the chain's ~3kn^2 at mid-depth, no free 2x.
Next step	Changing the tensor's factorisation is a dead end. What needs to change is what's being represented: compute the needed marginal channels (d3, variance, next-layer mean) without the kappa3 tensor.

D16 Channel-history state (m, C, Y, k4 for every layer) + nonlinear memory closure

Dead

Source: GPT-6 Astra (Round 11)

What to do	Dropping the source leg and keeping only the history of observable channels; a nonlinear rule for the next Y and k4.
Evidence / so far	Astra itself couldn't produce a rule (an honest 'no'). Injection argument: an all-distinct (a,b,c) hidden perturbation is invisible in the current slice and diagonal, yet it changes the next slice — so the channel alone doesn't generically carry enough information. In real MLPs: DHMP (tree from pair state) correlation 0.43-0.54, half the all-distinct part is memory; slice transport (K3g) and learned correction are already dead. Budget: 125u = ~15 square products per layer. | Astra (2nd answer, 24 Sep): again an honest 'no' — the S = (mu, C, Y21, d3, d4) state's O(n^2) storage and ~1u/layer covariance transport are fine, but the nonlinear transition for the missing cubic/quartic content is itself unknown; assuming it's free would make the result fabricated. Nothing new.
Next step	The injection test was never run: pass/fail is determined by construction (tautological), it says nothing about the real family.
File	Architecture Analysis/Round_11_Astra_Channel_State_Assessment.pdf

D17 Carrying only diagonal kappa3/kappa4 (d3, K4d) + Hadamard-power transport; low-rank(+diag) covariance; rank-1 kappa3

Dead

Source: Round 11: SciMaster 'matrix sketch', 'independent-cumulant', 'diagonal-channel transport'

What to do	Dropping the source leg and off-diagonal kappa3; next d3 = sum Q^3 d3 (+ covariance/tree term); covariance as diag + rank r; kappa3 ~ s t(x)t(x)t.
Evidence / so far	One-step teacher-forced (from the true post state): next-d3 error — independence assumption 90-99%, + dressed tree 83-98%, full repeated-index part (needs the off-diag slice) + tree 22-29% (chain free-run 3.2-4.7%). d3 comes almost entirely from off-diagonal kappa3. Best rank-1 D21: 35-49% error. Diag + rank-r covariance off-diag error at r=4/16/32/64: at L15, 65/43/27/14% (chain 0.7%). SciMaster's 'prototype at 1.18x MSE for 115u' — no run was ever done, a fabricated result.
File	experiments/probes/r11_diag_closure_gate.py

D18 **Carrying C, the Y21 slice, K4d, and a response matrix R (R' = (WoW) R W^T + alpha Y21 + beta C)** Dead

Source: Copilot (Round 11): Response-State Closure

What to do	Dropping the source; keeping the slice as state with a linear recursion + fitted coefficient.
Evidence / so far	Even under the most generous gate (true post state, in-sample fit at the same layer, L15->L16): off-diag D21 error 35.9%, d3 21.6% (chain 6.6% / 4.7%). The R memory term gains only 0.3%. No slice-to-slice linear recursion can capture all-distinct memory (same conclusion as K3g).
File	experiments/probes/r11_rsc_gate.py

D19 **Instead of a source bank, K = kM (~448) deterministic moment-matched particles; all channels from the cloud's empirical cumulants** Dead

Source: SciMaster (Round 11): Deterministic Particle Propagation / dNTC

What to do	Spherical design / cross-polytope particles, exact W and ReLU per particle; cost ~K/n u per layer.
Evidence / so far	Direct free-run (MLP #0, L15): cross-polytope 2048 — variance 3.0%, C off-diag 16%, d3 47%, K4d 183%, D21 74%; two-radius 4096 — nearly the same; 448 particles — variance 6.7%, D21 160%. Almost no difference between the deterministic design and random Gaussian (the design loses its advantage after the first ReLU). Chain: variance 0.14%, D21 6.6% — 20-50x worse. (Sampling was already dead.) dNTC's landmark part = Nyström/ISDF (dead).
File	experiments/probes/r11_particle_gate.py

D20 **Aggregate rank-k CP of old kappa3, transporting the factors via B** Dead

Source: Round 11: aggregated rank-k old kappa3 (T_old ~ sum sigma_a u_a x u_a x v_a, k ~ 230)

What to do	All old sources combined into one CP; transport exact per factor; D21 = (U o U) diag(sigma) V^T.
Evidence / so far	No new run needed, using previously measured evidence: (1) the pooled old tensor's (a)|(bc) unfolding needs rank ~250 to reach 2% (32/128/256 -> 35/7.8/1.6%) — this is a lower bound on CP rank, the true CP would need far more rank; (2) capping CP rank to 2% in the chain needed ~8n columns; (3) the forum's rank-4608 CP channel gives 1.07e-7 (5x worse than the chain). The claimed 'rank-230 gives 2%' is actually the matrix rank of the D21 slice, not the tensor's CP rank. Each layer adds ~3n of new-rank birth, and re-compression (factored CP-ALS) costs ~2u per sweep.
File	docs notes F88/K3 pooled-core rows

D21 Mixed forward/reverse or hierarchical contraction order for the young tier

Dead

Source: Sonnet 5 subagent (Round 10 A1, contraction-order search)

What to do	Writing the young tier as a tensor network and searching all multiplication orders (opt_einsum + hand-built mixed orders).
Evidence / so far	Reproducing the ledger: transport 54.5u (108 products), D21 hub 50.5u (100 products), total 105.0u. The Hadamard has to be built first, and the code already fuses 2 right-factor contractions — that's the floor. The transport chain's cost is order-independent (all n×n). Reverse-style suffix precompute +75%, per-age separation +150%, batching is FLOP-neutral. Correction: A = P·R_b, R_b a fixed, full (non-diagonal) n×n birth matrix — A and P transport under the same left map (matches to 1e-14); the earlier claim that 'P^-1 A changes by 100-970%' was an ill-conditioned-inverse error (one Round 10 answer caught this). No cost gain: building A entry-wise still costs n^3. | The 4 outside Round 10/10b answers agree: 193 square products (39 P + 54 A + 100 D21) = 98.4u core, no order is cheaper. Astra: float64 numerical packing — the meter already counts float64 as double, zero gain (float16 = float32 = 1.0, no precision discount). One answer assumed a wrongly-prompted diag(a_b) and claimed 'A never needs to be built' — not applicable. Another answer's 49-55u claim was flagged by a different validator as a miscalculation.
Next step	Making the young tier cheap needs a representation change, not a reordering.
File	experiments/probes/sonnet10_*.py; consultations/sonnet/ROUND10_sonnet_report.md

D22 Cheap per-layer rule for κ4 (cheapening the Astra kernel, scale-mixture, C⊗C), the B2/WK12 half-cost family, λ refit, B1 memory carrier

Dead

Source: Opus 5.5 subagent (Round 12 accuracy thread)

What to do	Searching for the source of the 15-25% accuracy gap to the leaderboard.
Evidence / so far	New ReLU physics captures only 1-4% of the true κ4 at depth (from L8; 1.02 sanity at L2). Carrying the true κ4 linearly gives level 83-90%, pattern correlation 0.77-0.80 (chain 0.26-0.34). The chain's κ4 structurally lacks the (1,1,1,1) all-distinct part. Scale-mixture/C⊗C correlation ≤0.09. FB 2.0 +3.8%/+3.2%, λ 0.85 +1.8%/+1.0% (MLP#0/synth). D21 oracle: ×0.9999 in the feedback leg alone (cross-talk is dead), ×0.758 everywhere (exact κ3 slice = -24%). B1: best ~2-product carrier misses 27% of D21 at L15; shortfall 40%, rank90 ~85, correlation with the previous step -0.0004. The 'κ4→κ3 feed arm' is already in the chain.
Next step	Q18
File	experiments/probes/opus12_*.py; consultations/fable/ROUND12_opus_report.md

Q18 κ4 memory horizon: how much of the pattern do the last 1-4 steps of birth capture?

Source: Opus 5.5 subagent (Round 12)

Dead

What to do	Extending opus12_k4_carry.py to carry the true κ4 linearly for only the last h steps (h=1..4); measuring pattern correlation and level.
Evidence / so far	κ4's deep pattern = carried memory (correlation 0.77-0.80 vs. chain 0.3); κ4 oracle -30%. | Result (q18_k4_horizon.py, N=1e6, MLP#0): erasing all memory at L-h and running the actual network forward h steps with Gaussian inputs, κ4 level at the decisive neuron, L12: h=1/2/3/4/6 → 2/6/12/18/35%, L16: 3/2/5/9/20%; pattern correlation with teacher L12 0.23/0.40/0.53/0.67/0.78, L16 0.20/0.41/0.59/0.65/0.77; best-scale errG4 h=2: 0.90/0.99, h=6: 0.37/0.63 — chain 0.19/0.21. The last 2 steps' births carry only 2-6% of κ4; even at 6 steps only 20-35%. κ4's memory is as long as κ3's: the young κ4 carrier is dead; the κ4 lever would need the full κ4 source chain — closed on cost grounds.
Cost	0u, ~30 minutes offline
Gate to pass	If the last 2 steps capture ≥ half the shortfall, build the young κ4 carrier (~4 products/layer ≈ 28u, +11%); ship only if raw ≥12% on both networks
File	experiments/probes/q18_k4_horizon.py

Q19 Refitting the disabled online mean correction (CORR_BETA) on v40fb's trajectory

Dead

Source: Own idea (after reading the 504aldo findings log F47, 25 Sep)

What to do	v40fb has an online mean correction but it's disabled (NO_CORR=1): the β fitted for the old V16b hurts on the new base, so it's 'off until refit' — the refit was never done. F47's method: on a teacher-forced trajectory (with the true mean from the previous layer substituted in), ridge-fit each layer's fresh error from 13 per-neuron features (mu, var, sigma, alpha, phi, Phi, pred, D3, |D21| row-sum, K4, ...), then apply it online inside the loop. Cost is near zero (13 dot products/neuron/layer).
Evidence / so far	F47: on the V16b base, 3.65e-8 -> 2.52e-8 = 1.45x (5 train / 3 test MLPs, train~test, generalizes). Our older 'learned OUTPUT correction dead' finding was about a correction outside the loop — this is different. Risk: v40fb's regen/vecww base may already capture part of this; the gain is unknown. | Result (25 Sep, MLP#0, experiments/variants/estimator_q19corr.py + experiments/chain/q19_run.py): with the correction left off, the variant matches the shipped result exactly (2.2364e-8). In the teacher-forced run, each layer's fresh-error rms is 1.5-6.7e-5; with the 13 features, in-sample R^2 on the same network is only L3 0.15, L8/12/15 0.02 — about equal to the 13-parameter / 1024-neuron chance level (~0.013), i.e. no signal (F47 had R^2 0.3-0.6 on the V16b base). v40fb's regen/vec+ww base has already absorbed the predictable part. Extra finding: forcing the mean to the true value drops final MSE from 2.24e-8 to 4.2e-9 — ~80% of the final error is accumulated mean error carried from earlier layers.
Cost	~0u (13 dot products/neuron/layer)
Gate to pass	If paired raw drops ≥3% on a held-out MLP, run Kaggle 100-MLP; fit only on local MLPs, never on the leaderboard
File	experiments/variants/estimator_q19corr.py; experiments/chain/q19_run.py

D23 Breaking the one-for-one rule by reallocating rank/age-limit by layer (more cost at later layers)

Dead

Source: Own idea (25 Sep)

What to do	Substituting the exact κ3 input (slice + d3) only in a specific layer window, to see where the gain accumulates.
Evidence / so far	Oracle (opus12_d21oracle.py + OPUS_OR_LMIN/LMAX, plain chain, MLP#0 / synthetic): all layers 0.608 / 0.715; layer ≥8 0.637 / 0.754; ≥12 0.704 / 0.863; ≥14 0.848 / 0.903; ≤11 0.884 / 0.837 (effects combine multiplicatively: 0.884×0.704≈0.62). The gain is spread across layers 8-16, not concentrated in the last 2-3 layers. And from an earlier measurement (notes K3ad/F-A5): even with the old sources kept fully dense and exact (no truncation at all), slice error only goes from 6.7% to 6.6%, final MSE only -3.9% (+19% cost) — the κ3 error doesn't come from compression, it's intrinsic to the birth of the K3 formulation (all-distinct post-ReLU κ3). So however rank/age-limits are reallocated, no large gain is possible.
File	experiments/variants/opus12_d21oracle.py; results/summaries/teacher_eval_q20_late_k3oracle.json

D24 V31c: add the sphere's remaining (2,2) kappa4 term 2c C_ij^2 at layer 0

Dead

Source: Own (V31 follow-up listed 22 Sep 07:06, dropped; found by the conversation audit 25 Sep)

What to do	The spherical input's layer-0 kappa4 (2,2) slice is c (C_aa C_bb + 2 C_ab^2), c = -2/(n+2); V31 carries only the first part in wk4m. V31c adds (2g/3) C_off^2 to wk4m at layer 0 (g = -6/(n+2)), with a scale knob V31C_S.
Evidence / so far	experiments/variants/estimator_v31c.py, paired on the two teacher networks (MLP #0 / synthetic): scale 1 +0.04% / +0.15%; 0.5 +0.10% / +0.14%; 2 +0.02% / +0.19%; -1 -0.12% / +0.11%. No consistent direction, noise level: the term (~1e-6 per entry) is too small to matter. Dead.
Cost	0 u
File	experiments/variants/estimator_v31c.py; results/summaries/teacher_eval_v31c.json

D25 Memory-saving Strassen level 6

Dead

Source: Own (promised 24 Sep 20:23 as a sure gain; found dropped by the conversation audit)

What to do	Run the shared-basis / young families one Strassen level deeper, computing block products step by step to stay under the 8 GB cap.
Evidence / so far	Earlier measurements: young-only L6 util -1.0% at 7.89-8.10 GB with 2x residual (21 Sep); MIN=6 -0.74% at 7.84 GB with +31% residual (23 Sep); level 6 on a 1024-side family spends ~half of its matmul saving on block additions. Best case < 1% cost for a large residual and memory risk against the 0.4 s / 8 GB caps. Decision 25 Sep: dropped.
File	docs notes K1/K2